

TEMA 1. Clases y objetos

1. ¿Cuáles son las cuatro características básicas de la programación orientada a objetos? Describe brevemente cada una

1. Abstracción

La abstracción consiste en identificar las características y comportamientos esenciales de un objeto, ignorando los detalles irrelevantes para el problema que se intenta resolver. Se trata de crear un modelo simplificado de la realidad. Por ejemplo, si se programa un sistema para una clínica, de una **Persona** solo interesan su **historialMedico** y **nombre**, ignorando otros datos como su color favorito o estatura.

En términos de implementación, se logra mediante la creación de clases que definen interfaces claras. Para un programador de C, la abstracción es similar a definir un archivo de cabecera (**.h**) donde se exponen solo las funciones necesarias para usar un módulo, ocultando la complejidad del código interno que el usuario no necesita conocer para que el sistema funcione.

2. Encapsulamiento

El encapsulamiento es el mecanismo que agrupa datos (atributos) y los métodos que los manipulan dentro de una misma unidad o "caja", restringiendo el acceso directo desde el exterior. El objetivo principal es proteger la integridad de los datos internos, evitando que el estado de un objeto sea modificado de manera arbitraria o incorrecta por otras partes del programa.

Para lograr esto en Java, se utilizan modificadores de acceso (como **private**). En lugar de acceder a una variable directamente como se haría en un **struct** de C, se emplean métodos intermedios (llamados *getters* y *setters*). Esto permite que el objeto tenga control total sobre sus propios datos, pudiendo validar una información antes de permitir que se guarde.

3. Herencia

La herencia es la capacidad de crear nuevas clases a partir de clases ya existentes. La nueva clase (clase hija) adopta automáticamente los atributos y métodos de la clase original (clase padre), pudiendo además añadir sus propias características o modificar las existentes. Es la herramienta principal para la reutilización de código y para establecer jerarquías lógicas.

A diferencia de C, donde para crear una estructura similar a otra se suele recurrir a copiar código o usar punteros complejos, en Java la herencia permite extender funcionalidades de forma natural. Por ejemplo, una clase **Vehículo** puede heredar sus propiedades básicas a una clase **Coche** y a una clase **Moto**, evitando tener que definir la matrícula o el motor por duplicado en cada una.

4. Polimorfismo

El polimorfismo es la característica que permite que un mismo nombre (ya sea un método o un objeto) se comporte de diferentes maneras según el contexto. En esencia, permite tratar objetos de diferentes clases hijas de manera uniforme si comparten una misma clase padre, ejecutando cada uno su propia versión específica del comportamiento.

Para un programador de C, esto sería comparable a tener un puntero genérico que sabe exactamente a qué función llamar sin necesidad de usar largas estructuras de `switch` o `if/else` para comprobar el tipo de dato. Si se tiene una lista de `Vehiculos` y se les ordena `arrancar()`, el polimorfismo garantiza que el coche use su motor de combustión y la bicicleta simplemente se ponga en marcha, todo bajo la misma instrucción.

2. Cita cuatro lenguajes populares que permitan la programación orientada a objetos

1. Java

Es el lenguaje referente en la enseñanza de la POO. A diferencia de C, donde el código se compila para un procesador específico, Java se compila para la **Máquina Virtual de Java (JVM)**, lo que le permite ejecutarse en cualquier sistema. Su sintaxis te resultará familiar, pero elimina la gestión manual de memoria (punteros y `free`) gracias al **Garbage Collector**.

En Java, la orientación a objetos es obligatoria: no puedes tener funciones sueltas "al aire" como haces en C; todo debe pertenecer a una clase. Es un lenguaje de tipado fuerte y estricto, diseñado para que los errores se detecten antes de ejecutar el programa.

2. C++

Es la evolución directa del C que ya conoces. Se considera un lenguaje **híbrido**, ya que permite mezclar la programación estructurada procedimental con la orientada a objetos. Esto lo hace extremadamente potente y eficiente, siendo el estándar en industrias donde el rendimiento es crítico, como los videojuegos o sistemas de alto tráfico.

La gran ventaja (y peligro) para alguien que viene de C es que mantienes el control de bajo nivel. Sigues teniendo punteros y gestión manual de memoria, pero ahora puedes encapsular esa lógica dentro de clases para organizar mejor proyectos grandes.

3. Python

Es un lenguaje de **alto nivel** con una sintaxis muy limpia que elimina la necesidad de llaves y puntos y coma. Aunque parece sencillo, es profundamente orientado a objetos: en Python, absolutamente todo (desde un número entero hasta una función) se trata internamente como un objeto.

A diferencia de C, Python es de **tipado dinámico**, lo que significa que no declaras el tipo de las variables (`int`, `float`, etc.). Esto permite desarrollar código mucho más rápido, aunque requiere más cuidado para evitar errores de tipo que en C se detectarían al compilar.

4. C# (C-Sharp)

Creado por Microsoft, es el lenguaje principal para el desarrollo en Windows y el motor de juegos Unity. Combina la elegancia y seguridad de Java con la potencia de C++. Su sintaxis es casi idéntica a la de Java, por lo que si aprendes uno, el otro te resultará muy intuitivo.

Al igual que Java, gestiona la memoria de forma automática. Es muy popular en el ámbito empresarial debido a su robustez y a la gran cantidad de herramientas que ofrece para crear aplicaciones de escritorio, web y móviles de forma organizada mediante objetos.

3. Los paradigmas anteriores a la POO, ¿Qué es la **programación estructurada**? y, todavía mejor, ¿Qué es la **programación modular**?

3. Paradigmas anteriores a la POO

Programación Estructurada

La programación estructurada es un paradigma que busca mejorar la claridad y calidad de un programa utilizando únicamente tres estructuras de control: **secuencia** (ejecución línea a línea), **selección** (**if/else**, **switch**) e **iteración** (bucles **for**, **while**). Este modelo nace para eliminar el uso del "salto incondicional" (*goto*), que hacía que el código fuera casi imposible de seguir en proyectos grandes.

En este paradigma, el programa se ve como una serie de pasos lógicos que se ejecutan de arriba hacia abajo. El énfasis está totalmente en la **lógica y el flujo de control**, tratando los datos como entidades separadas que simplemente pasan de una instrucción a otra. Es la base fundamental del lenguaje C estándar.

Programación Modular

La programación modular es el siguiente paso evolutivo, donde el programa se divide en partes independientes llamadas **módulos** (o funciones en C). Cada módulo se encarga de realizar una tarea específica y bien definida. La idea principal es "divide y vencerás": en lugar de tener un archivo **main.c** de tres mil líneas, separas la lógica en diferentes archivos o funciones que se llaman entre sí.

Para un programador de C, la modularidad es lo que aplicas cuando usas archivos **.h** y **.c** separados. Esto facilita mucho la depuración y permite que varios programadores trabajen en partes distintas del código sin estorbarse. Sin embargo, sigue teniendo un límite: aunque las funciones están separadas, los **datos** suelen seguir estando "sueltos" o globales, lo que puede provocar errores si una función modifica algo que no debería.

Comparativa rápida con la POO

- **Estructurada/Modular:** Te centras en **cómo** se hace la tarea (las funciones son las protagonistas). Los datos están "desprotegidos".
- **POO:** Te centras en **quién** hace la tarea (los objetos son los protagonistas). Los datos están "protegidos" dentro del objeto.

4. ¿Qué tres elementos definen a un objeto en programación orientada a objetos?

1. Estado (Atributos)

El estado representa los datos o la información que el objeto almacena en un momento determinado. Se define a través de los **atributos** (variables). En C, esto equivale exactamente a los campos de una estructura. Por ejemplo, en un objeto **Coche**, su estado podría estar definido por los valores: **color** = "Rojo", **velocidad** = 0 y **marcha** = 1.

El estado es dinámico; cambia a lo largo de la ejecución del programa cuando el objeto interactúa con otros o realiza acciones.

2. Comportamiento (Métodos)

El comportamiento define qué acciones puede realizar el objeto o cómo reacciona ante peticiones. Se implementa mediante **métodos** (funciones internas de la clase). A diferencia de C, donde pasas un **struct** como argumento a una función externa, en POO el comportamiento está "pegado" a los datos.

Siguiendo el ejemplo del **Coche**, su comportamiento serían métodos como **acelerar()**, **frenar()** o **cambiarMarcha()**. Estos métodos son los únicos autorizados, idealmente, para modificar el estado del objeto.

3. Identidad

La identidad es lo que permite distinguir a un objeto de otro, incluso si tienen el mismo estado. En C, dos variables tipo **int** con valor **5** son difíciles de distinguir conceptualmente; en POO, dos objetos **Coche** pueden ser rojos y estar parados, pero son dos entidades distintas en la memoria del ordenador.

Cada objeto tiene una dirección única en la memoria (una referencia). Gracias a la identidad, el programa sabe que al llamar a **miCoche.acelerar()**, solo debe subir la velocidad de ese coche específico y no la de todos los coches del sistema.

5. ¿Qué es una clase? ¿Es lo mismo que un objeto? ¿Qué es una instancia? ¿Todos los lenguajes orientados a objetos manejan el concepto de clase?

¿Qué es una clase?

Una clase es el **modelo, plano o plantilla** que define cómo será un objeto. En términos de C, es el equivalente a la definición de un **struct** en un archivo de cabecera: no ocupa espacio real en memoria para datos, solo describe qué variables (atributos) y qué funciones (métodos) tendrá ese tipo de dato. Es una entidad abstracta que existe solo en el código fuente.

¿Es lo mismo que un objeto?

No. La diferencia es la misma que hay entre el plano de una casa y la casa física construida.

- La **clase** es el concepto (ej. la clase **Coche** dice que todos los coches tienen matrícula).
- El **objeto** es la entidad real (ej. el coche con matrícula **1234-BBB** que está aparcado en tu garaje).

Mientras la clase se escribe una sola vez, puedes crear miles de objetos basados en esa misma clase, cada uno con sus propios datos.

¿Qué es una instancia?

"Instancia" es simplemente el nombre técnico que se le da a un objeto cuando se relaciona con su clase. Se dice que un objeto es una **instancia** de una clase. El proceso de crear un objeto en memoria se llama "instanciación". En C, sería el equivalente al momento en que declaras una variable de un **struct** y el sistema reserva memoria para ella; en ese instante, has creado una instancia de ese **struct**.

¿Todos los lenguajes POO manejan el concepto de clase?

Aunque la inmensa mayoría de los lenguajes populares (Java, C++, C#, Python) son **basados en clases**, no todos los lenguajes orientados a objetos las usan.

Existe una variante llamada **POO basada en prototipos**. El ejemplo más famoso es **JavaScript** (en sus versiones originales). En estos lenguajes, no existe un "plano" o clase; los objetos se crean clonando otros objetos ya existentes (prototipos) y añadiéndoles nuevas características sobre la marcha. Es un enfoque distinto, pero sigue siendo orientación a objetos.

6. ¿Dónde se almacenan en memoria los objetos? ¿Es igual en todos los lenguajes? ¿Qué es la **recolección de basura**?

En la mayoría de los lenguajes, los objetos se almacenan en una región de la memoria llamada **Heap** (montículo). A diferencia del **Stack** (pila), donde se guardan las variables locales y funciones con un tamaño fijo y tiempo de vida predecible, el Heap es un área mucho más grande y dinámica (el tamaño se decide en tiempo de ejecución). Lo que está en el heap vive más allá que el método o función donde se ha creado. Al crear un objeto, el sistema busca un bloque libre en el Heap, guarda allí los datos y devuelve una **referencia** (un puntero gestionado) que se almacena en el Stack para poder acceder a él.

Desventajas: hay que liberarla cuando ya no se necesita. Se haría de forma manual (más difícil) o de forma automática (por ejemplo, con el recolector de basura).

¿Es igual en todos los lenguajes?

No exactamente. En lenguajes como **Java** o **C#**, prácticamente todos los objetos van obligatoriamente al Heap. Sin embargo, en **C++**, el programador puede elegir: es posible crear un objeto en el Stack (como una variable local que desaparece al cerrar la función) o en el Heap (usando el operador **new**). La diferencia clave es que lo que está en el Stack es más rápido pero limitado, mientras que lo que está en el Heap permite que el objeto sobreviva más allá de la función donde se creó.

¿Qué es la recolección de basura (Garbage Collection)?

La recolección de basura es un proceso automático encargado de gestionar la memoria del Heap. Su función es identificar qué objetos ya no están siendo utilizados por el programa (porque no hay ninguna referencia apuntando a ellos) y liberar ese espacio de forma automática.

En lenguajes como **C**, el programador debe liberar la memoria manualmente (usando **free()**), lo que suele causar errores si se olvida hacerlo. En lenguajes con **Garbage Collector** (como Java o Python), el programador se despreocupa de la destrucción de los objetos, ya que el sistema operativo "limpia" periódicamente los datos que han quedado huérfanos.

Desventaja: problema de rendimiento, hay veces que pasa y no hay basura.

7. ¿Qué es un método? ¿Qué es la **sobrecarga de métodos**?

¿Qué es un método?

Un método es una función que está definida dentro de una clase y que determina el comportamiento de los objetos de esa clase. A diferencia de las funciones globales en C, un método tiene acceso implícito a los atributos del objeto que lo invoca. Conceptualmente, se puede ver como un mensaje que se le envía al objeto para que realice una tarea o devuelva una información sobre su estado interno.

En su estructura, un método consta de una cabecera (que incluye el nombre, el tipo de dato que devuelve y los parámetros) y un cuerpo (donde se escribe la lógica). La principal ventaja es que el código queda

encapsulado: el mundo exterior solo sabe qué hace el método, pero no necesita saber cómo está implementado internamente.

¿Qué es la sobrecarga de métodos?

La sobrecarga es la capacidad de definir, dentro de una misma clase, dos o más métodos que tengan exactamente el **mismo nombre pero distintos parámetros**. Se considera una forma de polimorfismo en tiempo de compilación. Para que el compilador pueda distinguirlos, la lista de argumentos debe variar en el número de parámetros, en sus tipos de datos o en el orden de los mismos.

Este recurso es muy útil para ofrecer diferentes maneras de realizar una misma acción según los datos disponibles. Por ejemplo, una clase `Calculadora` podría tener un método `sumar(int a, int b)` y otro `sumar(double a, double b)`. El lenguaje detectará automáticamente cuál ejecutar basándose en los valores que se le pasen, evitando tener que inventar nombres distintos como `sumarEnteros` o `sumarReales`.

8. Ejemplo mínimo de clase en Java, que se llame Punto, con dos atributos, x e y, con un método que se llame `calculaDistanciaAOrigen`, que calcule la distancia a la posición 0,0. Por sencillez, los atributos deben tener visibilidad por defecto. Crea además un ejemplo de uso con una instancia y uso del método

Definición de la clase

En Java, los atributos con **visibilidad por defecto** (package-private) no llevan ninguna palabra clave delante (ni `public` ni `private`). Para realizar cálculos matemáticos como la raíz cuadrada o la potencia, se utiliza la librería estándar `Math`.

```
public class Punto {
    // Atributos con visibilidad por defecto
    double x;
    double y;

    // Método para calcular la distancia al origen (0,0)
    public double calculaDistanciaAOrigen() {
        // La fórmula es: raíz cuadrada de (x^2 + y^2)
        return Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));
    }
}
```

Ejemplo de uso (Instanciación)

A diferencia de C++, en Java es obligatorio usar la palabra clave `new` para crear el objeto en el **Heap**. El punto `.` se utiliza tanto para asignar valores a los atributos como para invocar el método del objeto.

```
public class Principal {
    public static void main(String[] args) {
        // 1. Crear la instancia (reserva de memoria)
```

```
Punto miPunto = new Punto();

// 2. Asignar valores a los atributos
miPunto.x = 3.0;
miPunto.y = 4.0;

// 3. Uso del método
double distancia = miPunto.calculaDistanciaAOrigen();

// Mostrar resultado (equivalente al printf de C)
System.out.println("La distancia al origen es: " + distancia);
}
```

Notas para tu perfil (C/C++)

- **Sin punteros explícitos:** Aunque `miPunto` es técnicamente una referencia (un puntero), en Java no usas `->` para acceder a los miembros; siempre se usa el punto `..`.
- **Valores iniciales:** En Java, al crear un objeto con `new`, los atributos numéricos se inicializan automáticamente en `0` (a diferencia de C, donde tendrían "basura" de memoria si no los inicializas).

9. ¿Cuál es el punto de entrada en un programa en Java? ¿Qué es `static` y para qué vale? ¿Sólo se emplea para ese método `main`? ¿Para qué se combina con `final`?

¿Cuál es el punto de entrada en Java?

El punto de entrada es el método `public static void main(String[] args)`. Al igual que en C, donde la ejecución comienza en la función `main`, la Máquina Virtual de Java (JVM) busca esta firma específica para arrancar el programa. La principal diferencia es que en Java este método debe estar obligatoriamente **dentro de una clase**.

¿Qué es `static` y para qué vale?

El modificador `static` indica que un miembro (método o atributo) pertenece a la **clase en sí** y NO a una instancia (objeto) concreta.

- En el caso del `main`, es `static` porque la JVM necesita poder llamarlo antes de que se haya creado cualquier objeto en la memoria.
- Un método estático puede ejecutarse sin necesidad de usar el operador `new`. Para un programador de C, un método `static` es lo más parecido a una **función global** tradicional, ya que no depende del estado de ningún objeto para funcionar. Dentro del método `static` no puedo usar nada que no sea `static`, es decir no puedo usar `this` por ejemplo. `Integer.parseInt` es un método `static` por ejemplo. O también `Math.sqrt`. No necesito un `new math` o un `new integer`. No se debe abusar del `static`.

¿Sólo se emplea para el método `main`?

No. Se puede emplear en cualquier atributo o método que deba ser compartido por todos los objetos de esa clase.

- **Atributos estáticos:** Sirven para valores comunes, como un contador de cuántos objetos se han creado o una constante de configuración. Si un objeto cambia un atributo estático, el cambio es visible para todos los demás objetos de esa clase.
- **Métodos estáticos:** Se usan para funciones de utilidad que no necesitan datos de un objeto específico. Un ejemplo claro es `Math.sqrt()`; no necesitas crear un "objeto matemática" para calcular una raíz.

¿Para qué se combina con `final`?

La combinación `static final` se utiliza para definir **constantes** en Java.

- `static` asegura que la variable solo ocupe un lugar en memoria para toda la clase (ahorro de recursos).
- `final` impide que el valor sea modificado una vez asignado (equivalente al `const` de C o a un `#define`).

Es una práctica estándar para valores globales inmutables, como `static final double PI = 3.14159;`.

10. Intenta ejecutar un poco de Java de forma básica, con los comandos `javac` y `java`. ¿Cómo podemos compilar el programa y ejecutarlo desde línea de comandos? ¿Java es compilado? ¿Qué es la **máquina virtual**? ¿Qué es el *byte-code* y los ficheros `.class`?

¿Cómo compilar y ejecutar desde la línea de comandos?

Para poner en marcha un programa (por ejemplo, `Main.java`), se utilizan dos comandos distintos en la terminal:

1. **Compilación:** Se usa el comando `javac Main.java`. Si no hay errores, esto genera un nuevo archivo llamado `Main.class`.
2. **Ejecución:** Se usa el comando `java Main` (sin la extensión). Este comando lanza el programa y muestra el resultado en la consola.

¿Java es compilado?

Java se considera un lenguaje **híbrido** (compilado e interpretado). A diferencia de C, que se compila directamente a código máquina (instrucciones que el procesador entiende directamente), Java se compila a un paso intermedio. No genera un binario específico para Windows o Linux, sino un código universal.

¿Qué es el Byte-code y los ficheros `.class`?

El **byte-code** es el lenguaje intermedio que genera el compilador de Java. Es un conjunto de instrucciones optimizadas que no están diseñadas para ejecutarse en un procesador físico, sino en uno virtual. Este byte-code se guarda en los archivos con extensión `.class`.

La ventaja de esto es la **portabilidad**: un archivo `.class` compilado en un Mac funcionará exactamente igual en un PC con Windows o en un servidor Linux, siempre que tengan instalada la máquina virtual.

¿Qué es la Máquina Virtual (JVM)?

La **JVM (Java Virtual Machine)** es el programa que actúa como un "traductor" en tiempo real. Lee el byte-code de los archivos `.class` y lo traduce a las instrucciones específicas del procesador y sistema operativo.

que estés usando en ese momento.

Para un programador de C, es como si tuvieras un intérprete que lee tu código a medida que corre para asegurar que sea seguro y eficiente. Es gracias a la JVM que Java puede ofrecer servicios automáticos como el **Garbage Collector** que vimos antes, ya que ella es la que realmente gestiona la memoria mientras el programa se ejecuta.

11. En el código anterior de la clase **Punto** ¿Qué es **new**? ¿Qué es un **constructor**? Pon un ejemplo de constructor en una clase **Empleado** que tenga DNI, nombre y apellidos

¿Qué es **new**?

El operador **new** es el encargado de **instanciar** un objeto, es decir, de reservar espacio real en la memoria (específicamente en el *Heap*). En lenguajes como C, esto equivale a la función **malloc**, pero con una diferencia fundamental: **new** no solo reserva memoria, sino que también invoca automáticamente al **constructor** de la clase para inicializar los datos del objeto y devuelve una referencia para poder manejarlo.

¿Qué es un constructor?

Un constructor es un método especial que se ejecuta únicamente en el momento de crear un objeto. Su función principal es asegurar que el objeto nazca con un estado válido (inicializar sus atributos). Se distingue de los métodos normales porque **debe tener el mismo nombre que la clase** y no tiene ningún tipo de retorno (ni siquiera **void**).

Si no se define ningún constructor, Java crea uno por defecto sin parámetros, pero lo habitual es definir uno propio para obligar a que se pasen los datos necesarios desde el principio.

Ejemplo de constructor en la clase **Empleado**

A continuación se muestra cómo se definiría el constructor para una clase con los atributos solicitados. En el ejemplo se usa la palabra clave **this** para diferenciar los atributos de la clase de los parámetros que recibe el constructor.

```
public class Empleado {
    String dni;
    String nombre;
    String apellidos;

    // Constructor de la clase Empleado
    public Empleado(String dni, String nombre, String apellidos) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}
```

Ejemplo de uso:

```
// Al usar 'new', se pasan los valores que recibirá el constructor
Empleado nuevoEmpleado = new Empleado("12345678A", "Juan", "García Pérez");
```

12. ¿Qué es la referencia `this`? ¿Se llama igual en todos los lenguajes? Pon un ejemplo del uso de `this` en la clase `Punto`

¿Qué es la referencia `this`?

La referencia `this` es un puntero implícito que tiene cada objeto y que apunta **a sí mismo**. Se utiliza dentro de los métodos o constructores de una clase para referirse a los miembros (atributos o métodos) del objeto que está ejecutando el código en ese momento.

Su uso principal es resolver la **ambigüedad** cuando un parámetro de un método tiene el mismo nombre que un atributo de la clase. Sin el uso de `this`, el lenguaje daría prioridad a la variable local (el parámetro), haciendo imposible acceder al atributo del objeto.

¿Se llama igual en todos los lenguajes?

No, aunque el concepto es el mismo en casi todos los lenguajes orientados a objetos, el nombre varía:

- **Java, C++, C# y JavaScript:** Utilizan la palabra reservada `this`.
- **Python:** Utiliza `self` (y a diferencia de Java, debe pasarse explícitamente como primer parámetro en cada método).
- **Ruby y Swift:** Suelen usar `self`.
- **Visual Basic:** Utiliza `Me`.

Ejemplo de uso de `this` en la clase `Punto`

En el siguiente ejemplo, el constructor recibe dos parámetros llamados igual que los atributos (`x` e `y`). Usamos `this` para indicarle al programa: "guarda el valor del parámetro en el atributo de este objeto".

```
public class Punto {
    double x; // Atributo de la clase
    double y; // Atributo de la clase

    // Constructor con parámetros
    public Punto(double x, double y) {
        this.x = x; // this.x es el atributo; x es el parámetro
        this.y = y; // this.y es el atributo; y es el parámetro
    }

    public void imprimirPosicion() {
        // También se puede usar para dejar claro que accedemos a datos del objeto
        System.out.println("Punto en: " + this.x + ", " + this.y);
    }
}
```

13. Añade ahora otro nuevo método que se llame `distanciaA`, que reciba un `Punto` como parámetro y calcule la distancia entre `this` y el punto proporcionado

Implementación del método `distanciaA`

Para calcular la distancia entre dos puntos en un plano cartesiano, se utiliza la fórmula de la distancia euclídea. En este caso, el método toma las coordenadas del objeto actual (accesibles mediante `this`) y las del objeto pasado como parámetro (accesibles mediante la referencia del parámetro).

Al venir de C, puedes observar que aquí tratamos con dos instancias de la misma clase simultáneamente: la que ejecuta el método y la que se recibe por argumento.

```
public class Punto {
    double x;
    double y;

    // ... (constructor y otros métodos)

    /**
     * Calcula la distancia entre el punto actual (this)
     * y otro punto recibido por parámetro.
     */
    public double distanciaA(Punto otroPunto) {
        // Fórmula: raíz cuadrada de ((x2 - x1)^2 + (y2 - y1)^2)
        double diferenciaX = otroPunto.x - this.x;
        double diferenciaY = otroPunto.y - this.y;

        return Math.sqrt(Math.pow(diferenciaX, 2) + Math.pow(diferenciaY, 2));
    }
}
```

Ejemplo de uso

En el programa principal, se deben crear dos instancias diferentes para poder calcular la distancia entre ellas.

```
public class Principal {
    public static void main(String[] args) {
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(3, 4);

        // Se pide a p1 que calcule la distancia hasta p2
        double resultado = p1.distanciaA(p2);

        System.out.println("La distancia entre los puntos es: " + resultado);
    }
}
```

```
}

```

14. El paso del **Punto** como parámetro a un método, es **por copia** o **por referencia**, es decir, si se cambia el valor de algún atributo del punto pasado como parámetro, dichos cambios afectan al objeto fuera del método? ¿Qué ocurre si en vez de un **Punto**, se recibiese un entero (**int**) y dicho entero se modificase dentro de la función?

Esta es una de las preguntas más críticas para alguien que viene de **C**, ya que la gestión de punteros en Java es automática pero sigue reglas muy estrictas que pueden confundir al principio.

14. Paso por valor vs. Paso por referencia

Paso de objetos (como **Punto**)

En Java, los objetos siempre se pasan por **referencia** (técnicamente, se pasa por valor la dirección de memoria del objeto). Esto significa que si pasas un objeto **Punto** a un método y modificas sus atributos dentro, **los cambios afectarán al objeto original** fuera del método.

Al recibir el parámetro, el método no crea una copia de los datos del punto, sino que recibe una "flecha" que apunta al mismo espacio de memoria (en el *Heap*) que el objeto original. Es el comportamiento equivalente a pasar un puntero a una estructura en C (**Punto *p**).

Paso de tipos primitivos (como **int**)

A diferencia de los objetos, los tipos primitivos (**int**, **double**, **boolean**, **char**, etc.) se pasan siempre **por valor** (copia). Si pasas un **int** a un método y lo modificas dentro, **el valor original fuera del método no cambia**.

En este caso, Java crea una copia exacta del dato en una nueva posición de memoria dentro del *Stack* del método. Al terminar la ejecución de la función, esa copia desaparece y la variable original permanece intacta. Esto equivale a pasar una variable normal en C sin usar el operador **&**.

Comparativa técnica

Tipo de dato	¿Cómo se pasa?	¿Cambia el original?	Equivalente en C
Objeto (Punto , String)	Referencia	Sí	<code>void func(Struct *p)</code>
Primitivo (int , float)	Valor (Copia)	No	<code>void func(int n)</code>

15. ¿Qué es el método **toString()** en Java? ¿Existe en otros lenguajes? Pon un ejemplo de **toString()** en la clase **Punto** en Java

¿Qué es el método **toString()**?

En Java, el método **toString()** es un método especial que devuelve una **representación en cadena de texto (String)** de un objeto. Todos los objetos en Java heredan este método de una clase base llamada **Object**.

Por defecto, si no se modifica, `toString()` devuelve algo poco útil como el nombre de la clase seguido de un código extraño (el hashCode). Sin embargo, lo habitual es **sobrescribirlo** para que devuelva información legible sobre el estado del objeto (sus atributos). Es extremadamente útil para tareas de depuración (debugging) y para mostrar información por consola sin tener que imprimir los atributos uno por uno.

¿Existe en otros lenguajes?

Sí, la mayoría de los lenguajes orientados a objetos tienen un concepto equivalente para convertir objetos en texto de forma automática:

- **Python:** Se utiliza el método especial `__str__` o `__repr__`.
- **C#:** Se llama exactamente igual, `ToString()`.
- **JavaScript:** También utiliza `toString()`.
- **C++:** Aunque no existe un método único, lo habitual es sobrecargar el operador de inserción de flujo `<<`.

Ejemplo de `toString()` en la clase `Punto`

Al definirlo, usamos la anotación `@Override` para indicar que estamos sustituyendo la versión por defecto de Java por la nuestra.

```
public class Punto {
    double x;
    double y;

    // Constructor...

    @Override
    public String toString() {
        // Devolvemos una cadena con el formato (x, y)
        return "Punto[x=" + this.x + ", y=" + this.y + "]";
    }
}
```

Ejemplo de uso:

```
Punto p = new Punto(5, 10);
// Java llama automáticamente a toString() al imprimir el objeto
System.out.println(p);
// Salida: Punto[x=5.0, y=10.0]
```

16. Reflexiona: ¿una clase es como un `struct` en C? ¿Qué le falta al `struct` para ser como una clase y las variables de ese tipo ser instancias?

¿Es una clase como un `struct` de C?

La respuesta corta es **sí, pero potenciado**. Una clase es la evolución natural del **struct**. Ambos son tipos de datos definidos por el usuario que sirven para agrupar información que tiene sentido que vaya junta (por ejemplo, los datos de un **Usuario** o las coordenadas de un **Punto**).

En C, el **struct** es puramente **pasivo**: es solo un contenedor de datos. En Java, la clase es **activa**: no solo guarda datos, sino que sabe cómo procesarlos.

¿Qué le falta al **struct** para ser una clase?

Para que un **struct** se convierta en una clase, le faltan principalmente tres componentes fundamentales:

1. **Comportamiento (Métodos) dentro de la estructura:** En C, las funciones están fuera. Si quieres mover un **struct Punto**, creas una función **mover(Punto *p)**. En una clase, la función está dentro. El dato y la lógica están "pegados" (Encapsulamiento).
2. **Modificadores de acceso:** En un **struct**, cualquiera que tenga acceso a la variable puede cambiar sus campos internos. En una clase, puedes poner "candados" (**private**) para que nadie toque los datos si no es a través de los métodos que tú permitas.
3. **Herencia y Polimorfismo:** Un **struct** no puede heredar de otro. No puedes decir que un **struct Coche** es una versión extendida de un **struct Vehiculo**. Las clases permiten crear estas jerarquías para reutilizar código.

¿Cuándo las variables de ese tipo pasan a ser instancias?

En C, cuando declaras **struct Punto p;**, el compilador reserva memoria y ya puedes usarlo. En Java, la variable **Punto p;** es solo un **puntero vacío** (una referencia).

La variable se convierte en una **instancia** realmente cuando usas la palabra clave **new**. En ese momento:

- Se reserva memoria en el **Heap**.
- Se ejecuta el **Constructor** (que no existe en los structs de C).
- El objeto adquiere su propia **Identidad** única en el sistema.

17. Qitemos un poco de magia a todo esto: ¿Como se podría "emular", con **struct** en C, la clase **Punto**, con su función para calcular la distancia al origen? ¿Qué ha pasado con **this**?

Esta es la pregunta definitiva para "romper" la magia de Java y entender que, en el fondo, el ordenador sigue funcionando de forma similar. Para emular la POO en C, tenemos que hacer manualmente lo que Java hace automáticamente por nosotros.

17. Emulando una Clase en C

La emulación del código

Para que un **struct** en C se parezca a una clase con métodos, necesitamos usar **punteros a funciones**. En C, los datos y las funciones están separados, así que debemos "atar" la función al struct manualmente.

En C se haría así:

```

#include <stdio.h>
#include <math.h>

// Definición del "molde" (la Clase)
struct Punto {
    double x;
    double y;
    // Puntero a función para emular el método
    double (*calculaDistanciaAOrigen)(struct Punto*);
};

// La función lógica que hará el trabajo
double funcionDistancia(struct Punto* p) {
    return sqrt(pow(p->x, 2) + pow(p->y, 2));
}

int main() {
    // Instanciación manual
    struct Punto miPunto;
    miPunto.x = 3.0;
    miPunto.y = 4.0;

    // "Conectamos" el método al dato
    miPunto.calculaDistanciaAOrigen = &funcionDistancia;

    // Uso del "método"
    double d = miPunto.calculaDistanciaAOrigen(&miPunto);
    printf("Distancia: %f", d);

    return 0;
}

```

¿Qué ha pasado con `this`?

En Java, `this` es "mágico" y aparece solo, pero en la emulación en C vemos la realidad: **`this` es simplemente un parámetro extra.**

Fíjate en la llamada: `miPunto.calculaDistanciaAOrigen(&miPunto)`. Para que la función sepa qué datos procesar, tenemos que pasarle explícitamente la dirección de la estructura (`&miPunto`).

- En **C**, tú pasas el puntero manualmente como primer argumento.
- En **Java**, el compilador lo pasa por ti de forma invisible y le pone el nombre `this`.

Conclusión

Lo que Java llama "Objeto" es, en esencia, una estructura de datos que lleva consigo un puntero a una tabla de funciones. La "magia" de la POO consiste en que el lenguaje se encarga de gestionar esos punteros y de pasar el valor de `this` automáticamente cada vez que llamas a un método, evitando que tú cometas errores al gestionar las direcciones de memoria.